bcrypt take a tunable *cost factor* to specify how expensive the hash computation will be. If your code defines this number as a constant:

13-understanding-test-doubles/03/stubbed_constants.rb

```
class PasswordHash
  COST_FACTOR = 12

  # ...
end
```

…your specs can redefine it to 1:

13-understanding-test-doubles/03/stubbed_constants.rb

```
stub_const('PasswordHash::COST_FACTOR', 1)
```

You can use stub_const to do a number of things:

- Define a new constant
- Replace an existing constant
- Replace an entire module or class (because these are also constants)
- Avoid loading an expensive class, using a lightweight fake in its place

Sometimes, controlling your test environment means *removing* an existing constant instead of stubbing one. For example, if you're writing a library that works either with or without ActiveRecord, you can hide the ActiveRecord constant for a specific example:

13-understanding-test-doubles/03/stubbed_constants.rb

```
hide_const('ActiveRecord')
```

Hiding the ActiveRecord constant like this will cut off access to the entire module, including any nested constants like ActiveRecord::Base. Your code won't be able to accidentally use ActiveRecord. Just as with partial doubles, any constants you've changed or hidden will be restored at the end of each example.

Your Turn

In this chapter, we discussed the differences between stubs, mocks, spies, and null objects. In particular, you saw how they deal with the following situations:

- Receiving expected messages
- Receiving unexpected messages
- Not receiving expected messages

We also looked at the different ways to create test doubles. Pure doubles are entirely fake, whereas partial doubles are real Ruby objects that have fake behavior added. Verifying doubles fall in between, and have the advantages of both with few of the downsides of either. They're the ones we use most often.